

Healthy side of functional programming

Bartłomiej Górny

bartlomiej.gorny@erlang-solutions.com



CODER'S HEALTH ISSUES

- ▶ Wrists
- ▶ Spine
- ▶ Eyes
- ▶ stress

CODER'S HEALTH ISSUES

- ▶ Wrists
 - ▶ Spine
 - ▶ Eyes
 - ▶ stress
-

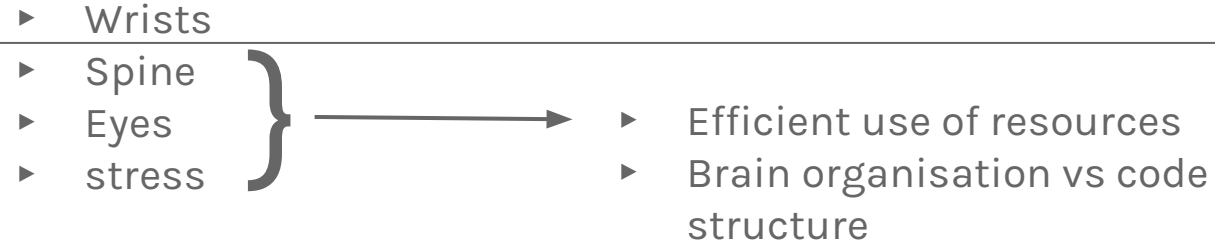
CODER'S HEALTH ISSUES

- ▶ Wrists
 - ▶ Spine
 - ▶ Eyes
 - ▶ stress
-
- }

CODER'S HEALTH ISSUES

- ▶ Wrists
 - ▶ Spine
 - ▶ Eyes
 - ▶ stress
-
- } → ▶ Efficient use of resources

CODER'S HEALTH ISSUES



WRISTS AND FINGERS

- ▶ RSI, CTS
- ▶ Keyboards
- ▶ Trackball
- ▶ Less typing

DOES FP MEAN LESS TYPING?

- ▶ QuickCheck implementations in various languages
 - ▷ (in thousands of lines)

Clojure	6.7
Haskell	9.6
C	12.2
C++	14.2
Java	17.9
JavaScript	18.5
PHP	20.1
.NET	77.4

MAKING WORK EASIER

- ▶ Use brain efficiently
 - ▷ Reduce effort and stress
- ▶ What are we best at?

MAKING WORK EASIER

- ▶ Use brain efficiently
 - ▷ Reduce effort and stress
- ▶ What are we best at?

PATTERN RECOGNITION

MAKING WORK EASIER

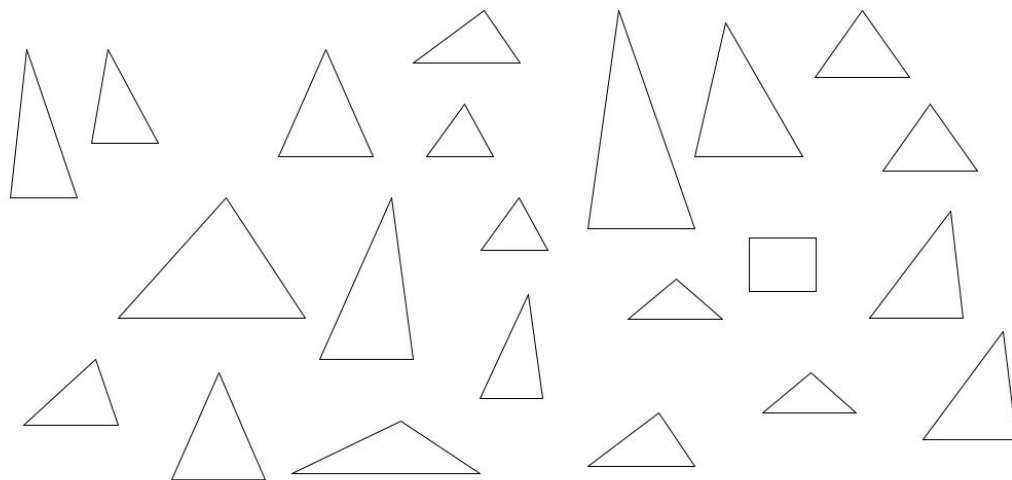
- ▶ Use brain efficiently
 - ▷ Reduce effort and stress
- ▶ What are we best at?

PATTERN RECOGNITION

- ▶ Evolutionary optimisation
 - ▷ Finding animal tracks
 - ▷ Searching edible plants
 - ▷ Recognising expressions (hostile or friendly?)
 - ▷ Danger alerts

PATTERN RECOGNITION

Can you spot what's wrong with this picture?



How long did it take you?

PATTERN RECOGNITION / MATCHING

```
def decide(good, bad, ugly):  
    if good == True and bad == False:  
        shoot()  
    elif bad == True and ugly == True:  
        hang()  
    elif good == False and ugly == False:  
        check()  
    elif bad == False and ugly == False:  
        fail("bad is always ugly")  
    elif good == True and bad == True:  
        fail("contradiction")  
    else:  
        pass
```

```
decide(true, false, _) ->  
    shoot();  
  
decide(true, _, true) ->  
    hang();  
  
decide(false, _, false) ->  
    check();  
  
decide(_, false, false) ->  
    fail("bad is always ugly");  
  
decide(true, false, _) ->  
    fail("contradiction");  
  
decide(_, _, _) ->  
    ok.
```

PATTERN RECOGNITION - OTHER EXAMPLES

- ▶ Chess
 - ▷ Analysis, strategic planning, forecasting
 - ▷ Recognising patterns on board
 - ▷ RPD (Recognition Primed Decisions)

RECOGNITION PRIMED DECISIONS

- ▶ Based on matching input to an in-memory pattern
- ▶ Done instantly without conscious thinking
- ▶ Often based on episodic patterns

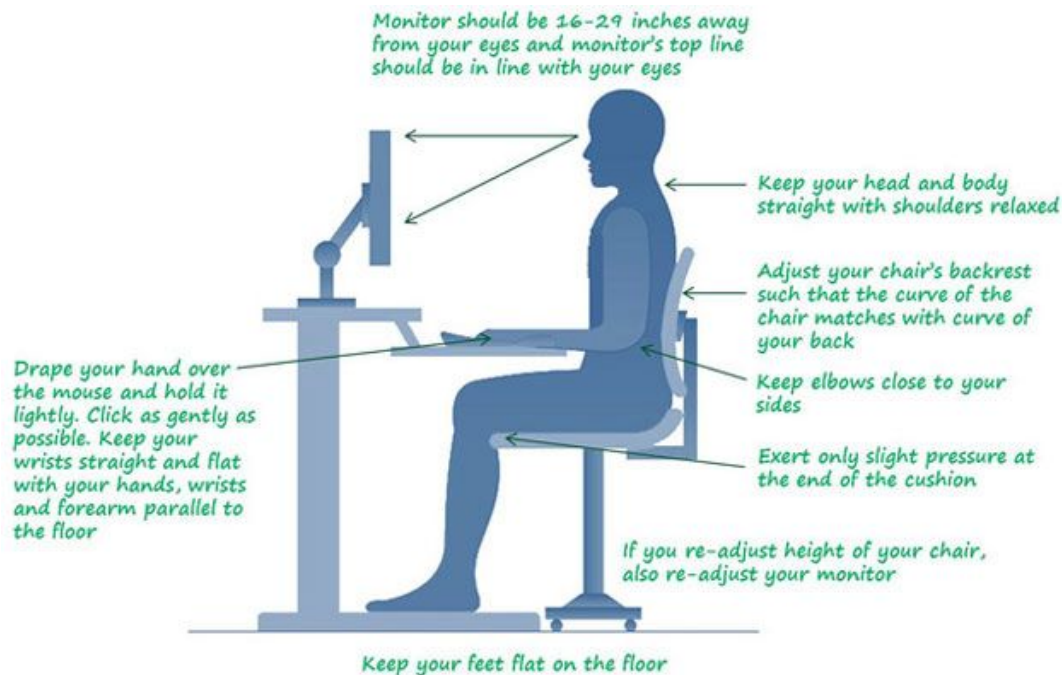
RECOGNITION PRIMED DECISIONS

- ▶ Based on matching input to an in-memory pattern
 - ▶ Done instantly without conscious thinking
 - ▶ Often based on episodic patterns
-
- ▶ ...some call it “experience”

IN SEARCH FOR THE RIGHT POSTURE

- ▶ ...and how FP can help with that

SPINE - THEORY



PRACTICE - COMMON TYPING POSITION



COMMON CODE READING POSITION



THINKING

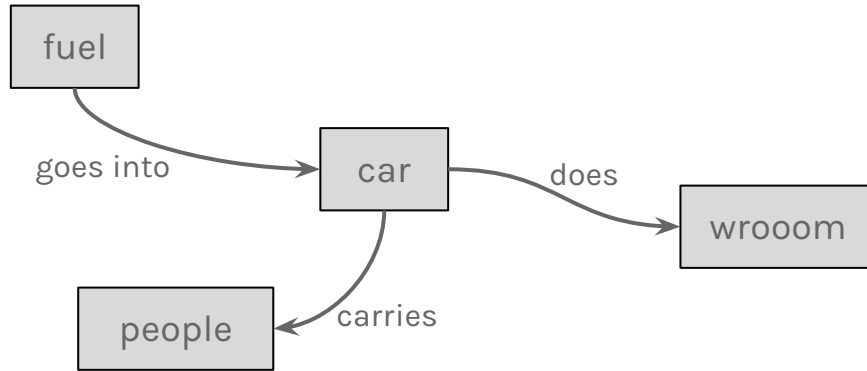


READING AND UNDERSTANDING

- ▶ We do much more reading than writing
 - ▷ Legacy code, morning bootstrap, code reviews...
- ▶ We have to understand the code

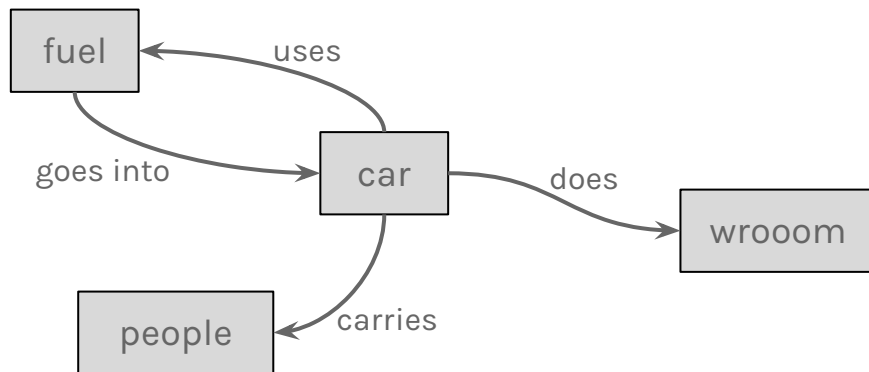
READING AND UNDERSTANDING

- To understand = to build a concept network



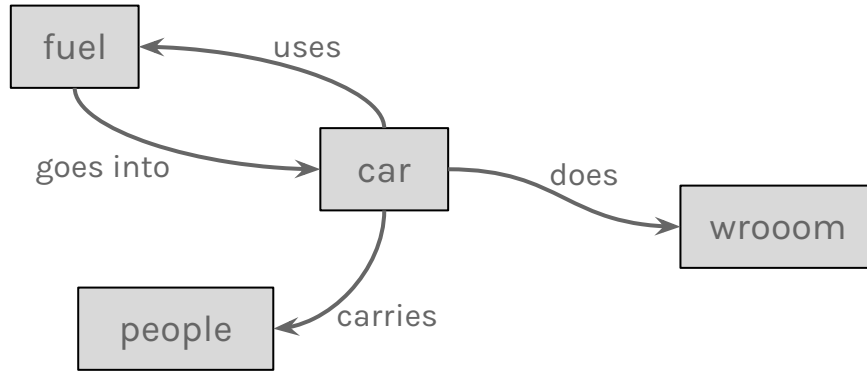
READING AND UNDERSTANDING

- To understand = to build a concept network



READING AND UNDERSTANDING

- ▶ To understand = to build a concept network



- ▶ Thinking is (re)building concept network

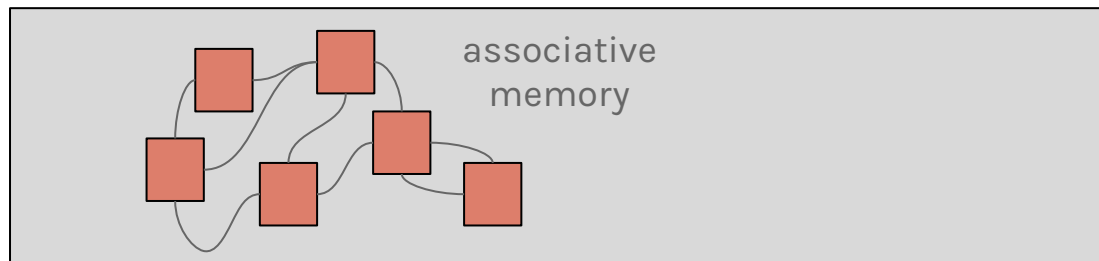
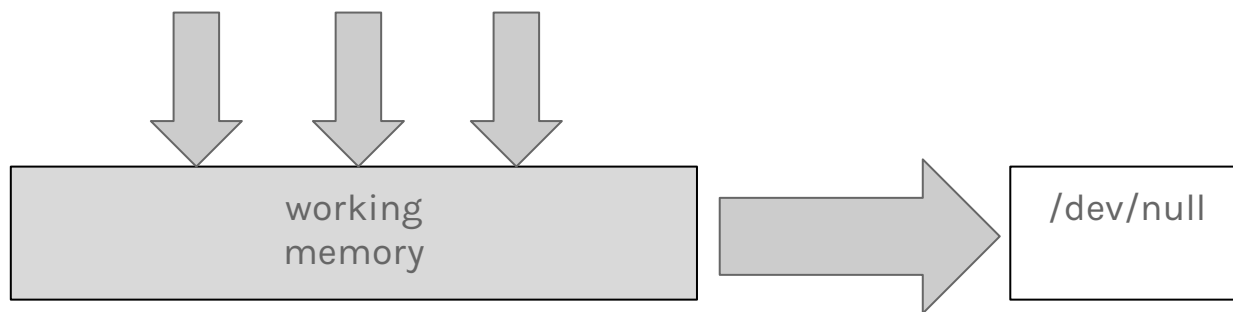
WHERE DO CONCEPTS COME FROM?

- ▶ Brain processes a stream of input
- ▶ Concepts must be:
 - ▷ Isolated
 - ▷ Formulated
 - ▷ stored

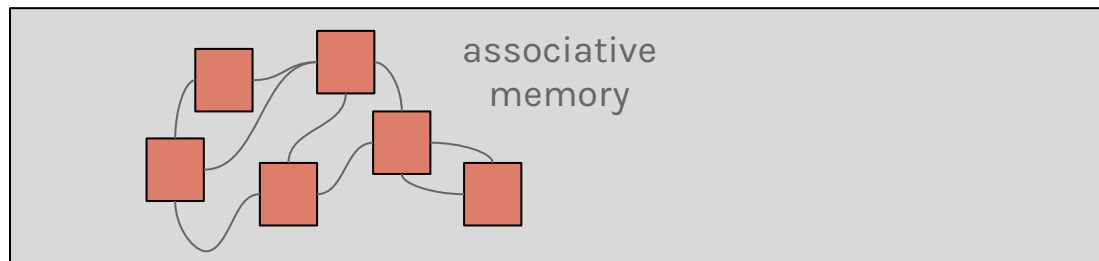
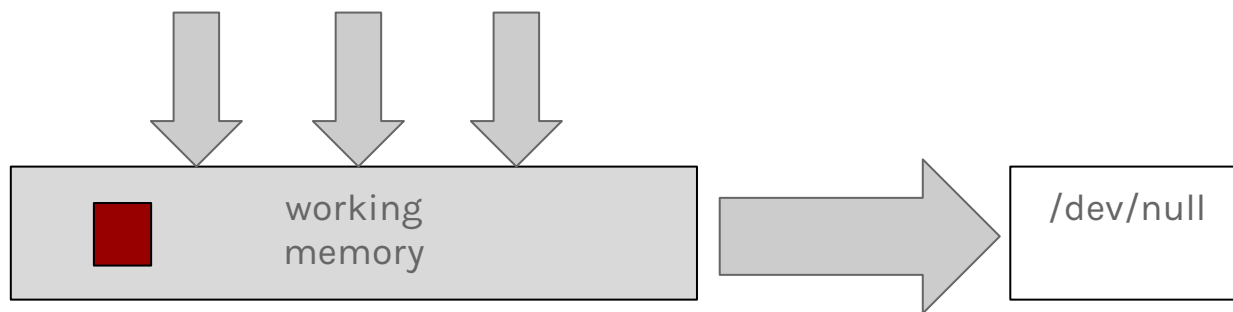
MEMORY STRUCTURE

- ▶ Working memory
 - ▷ “Input buffer”
 - ▷ High throughput
 - ▷ Low capacity
 - ▷ Volatile
- ▶ Associative memory
 - ▷ Low throughput
 - ▷ Unknown (infinite?) capacity
 - ▷ persistent

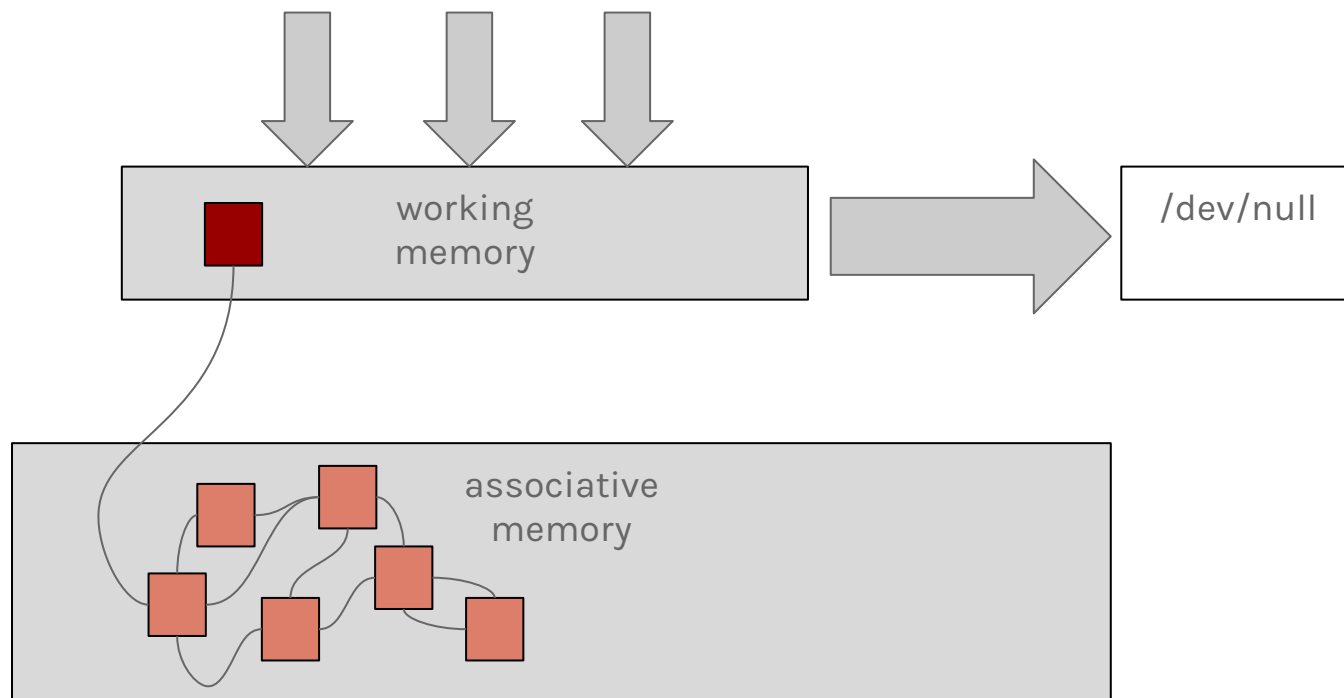
MEMORY STRUCTURE



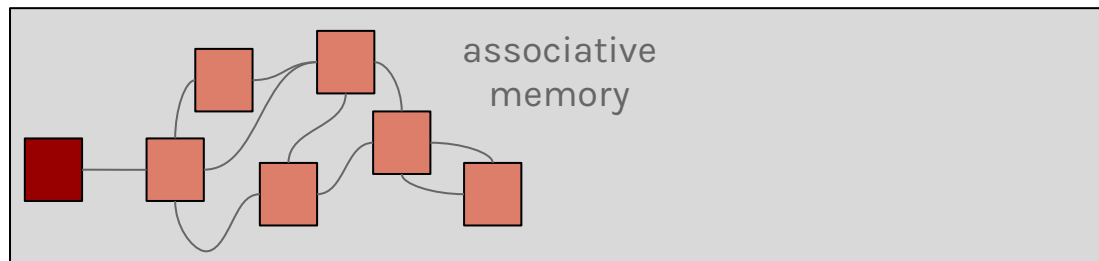
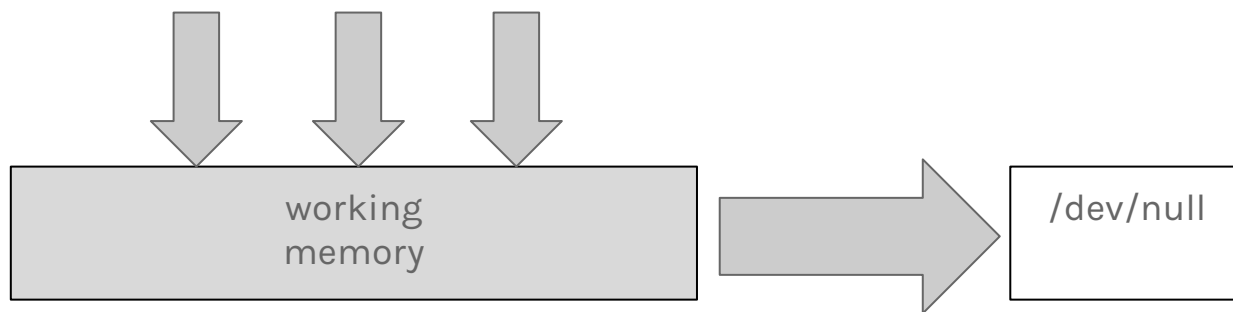
MEMORY STRUCTURE



MEMORY STRUCTURE



MEMORY STRUCTURE



ASSOCIATIONS

75

ASSOCIATIONS

75



ASSOCIATIONS IN ACTION

- ▶ Memorising a phone number (a tragic story)

ASSOCIATIONS IN ACTION

- ▶ Memorising a phone number (a tragic story)

687583861

ASSOCIATIONS IN ACTION

- ▶ Memorising a phone number (a tragic story)

ASSOCIATIONS IN ACTION

- ▶ Memorising a phone number (a tragic story)

68 75 8 386 1

ASSOCIATIONS IN ACTION

- ▶ Memorising a phone number (a tragic story)

68 **75** 8 386 1

ASSOCIATIONS IN ACTION

- ▶ Memorising a phone number (a tragic story)

68 75 **8** 386 1

ASSOCIATIONS IN ACTION

- ▶ Memorising a phone number (a tragic story)

68 75 8 **386** 1

ASSOCIATIONS IN ACTION

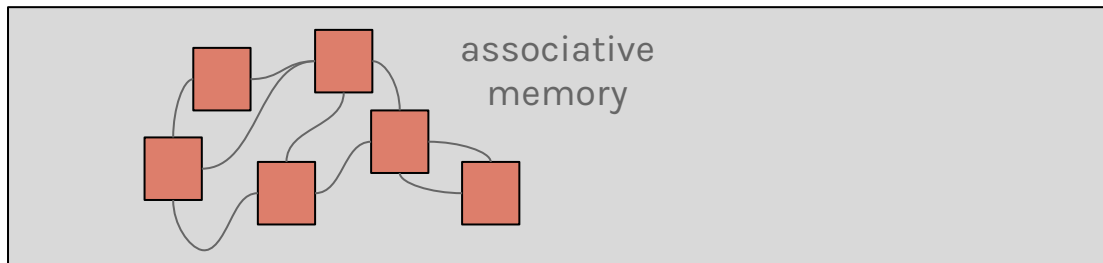
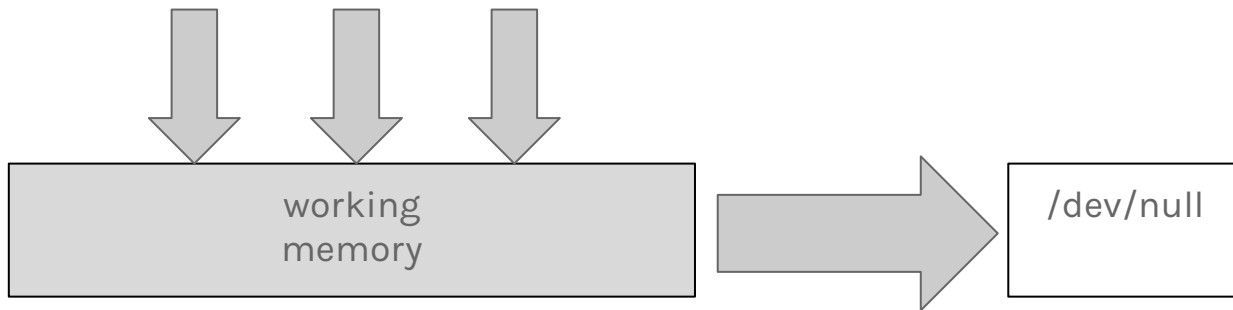
- ▶ Memorising a phone number (a tragic story)

68 75 8 386 **1**

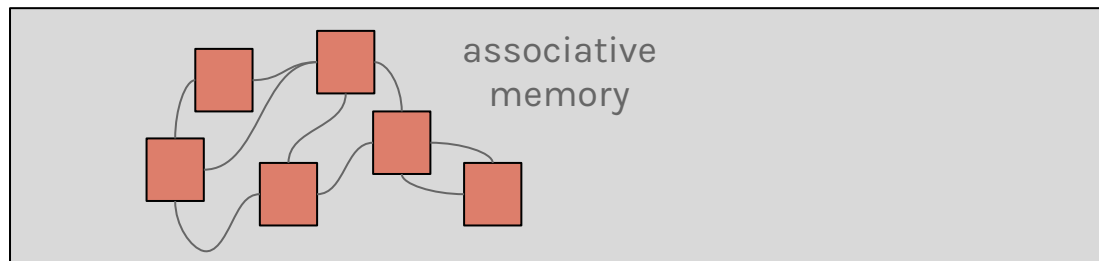
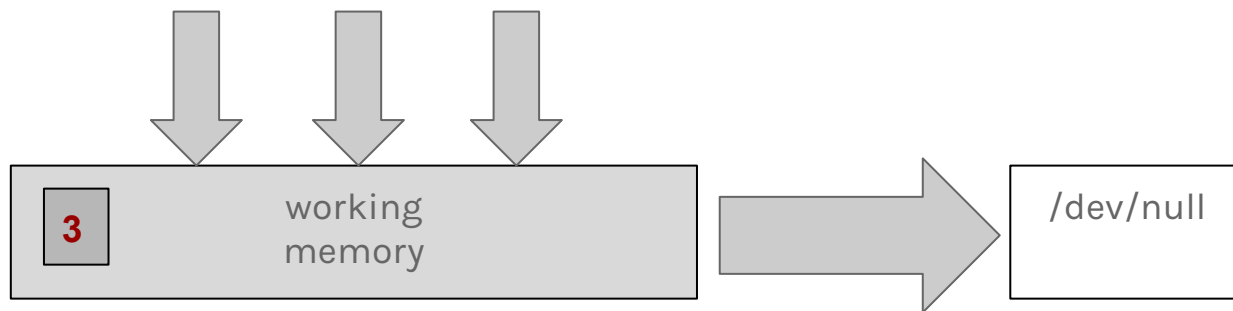
ASSOCIATIONS IN ACTION

- ▶ Memorising a phone number (a tragic story)

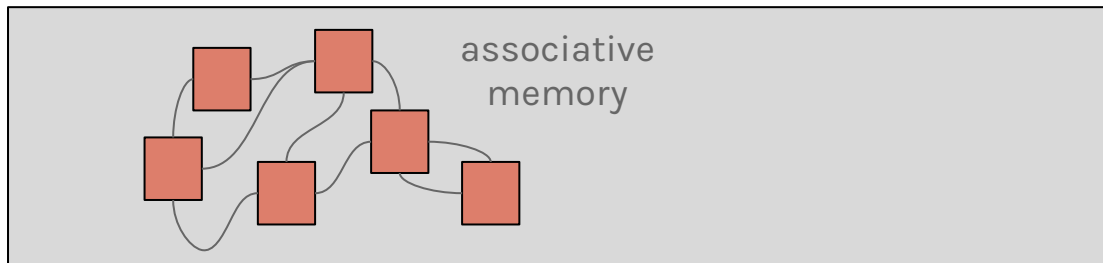
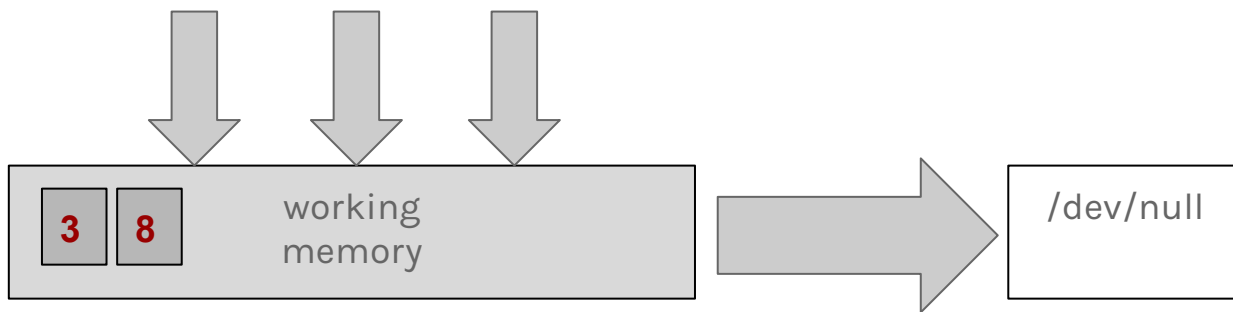
FROM INPUT TO CONCEPT



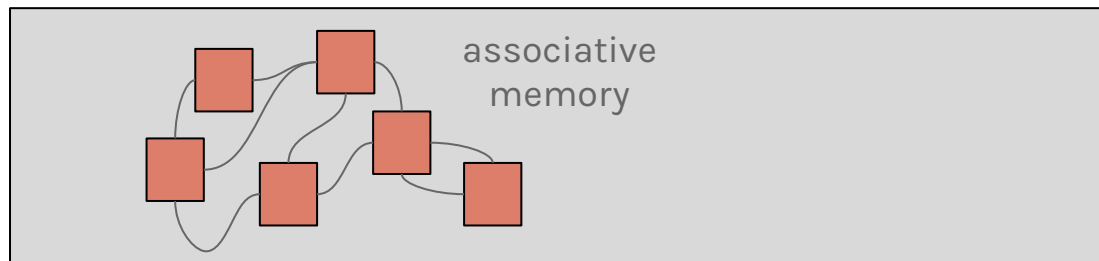
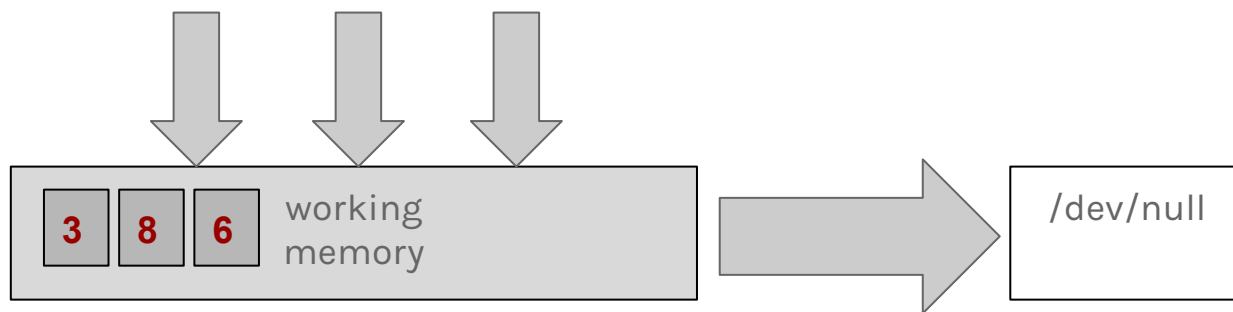
FROM INPUT TO CONCEPT



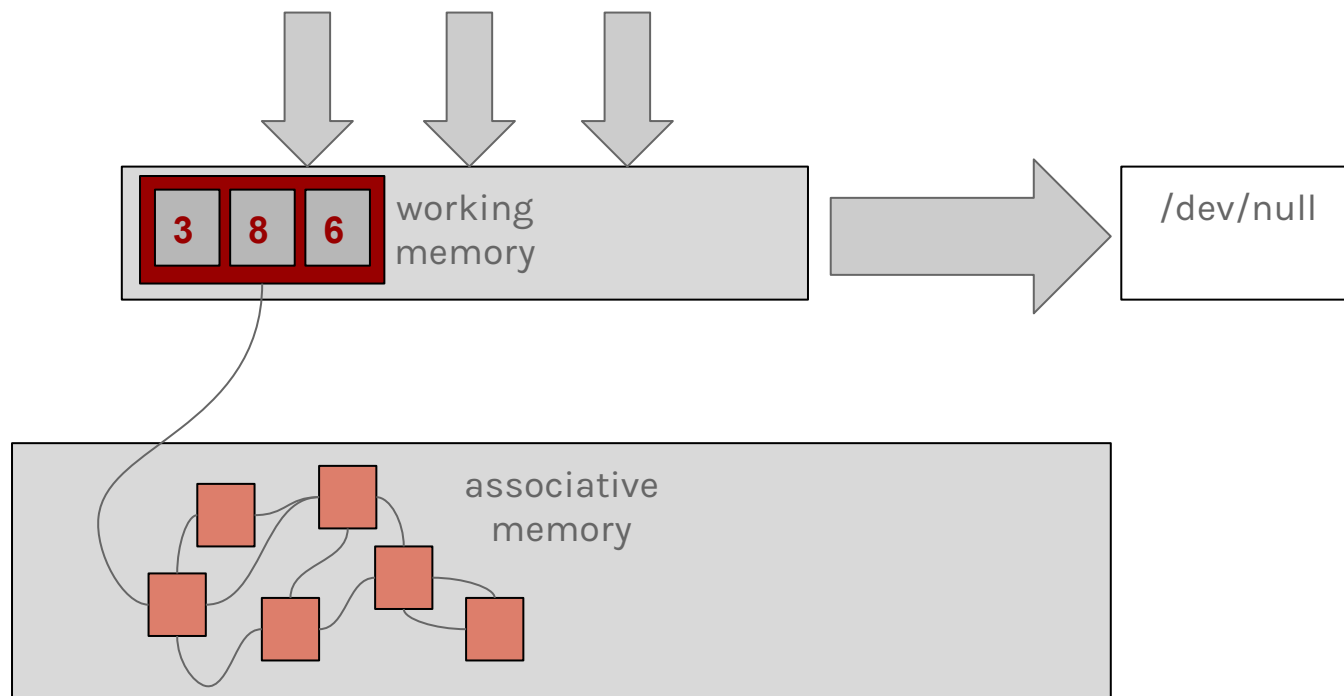
FROM INPUT TO CONCEPT



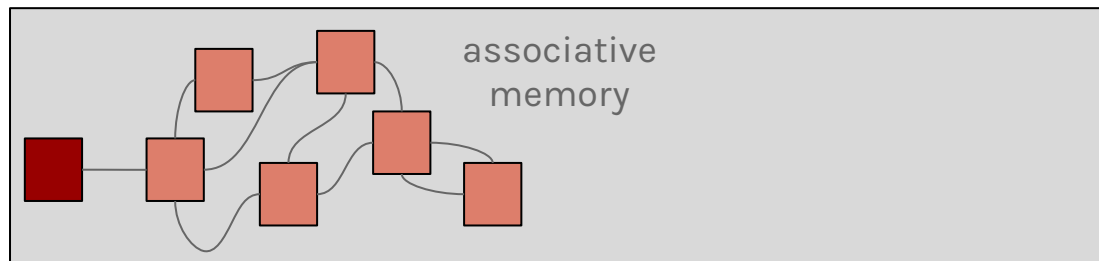
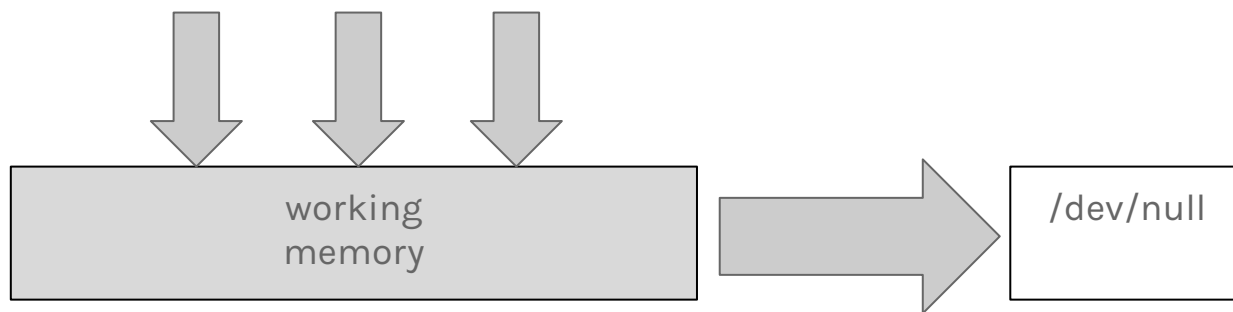
FROM INPUT TO CONCEPT



FROM INPUT TO CONCEPT



FROM INPUT TO CONCEPT



READING CODE, REVISITED

```
f = open("data.txt")
res = []
for line in f.readlines():
    lineparts = line.split(' ')
    line_1 = []
    for part in line:
        if part == '':
            part = None
        else:
            try:
                part = int(part)
            except ValueError:
                part = None
        if part != None:
            line_1.append(int(part))
    res.append(line_1)
```

```
let readarr =
    In_channel.fold_lines (open_in "data.txt")
    ~init:[]
    ~f:parserow;;

let parserow lst s =
    let n = String.split s ~on:' '
    |> List.filter_map ~f:makeint in
    (lst @ n)
;;

let makeint = function
| "" -> None
| s -> Some(Int.of_string s)
;;
```

READING CODE, REVISITED

```
f = open("data.txt")
res = []
for line in f.readlines():
    lineparts = line.split(' ')
    line_1 = []
    for part in line:
        if part == '':
            part = None
        else:
            try:
                part = int(part)
            except ValueError:
                part = None
        if part != None:
            line_1.append(int(part))
    res.append(line_1)
```

```
let readarr =
    In_channel.fold_lines (open_in "data.txt")
    ~init:[]
    ~f:parserow;;

let parserow lst s =
    let n = String.split s ~on:' '
    |> List.filter_map ~f:makeint in
    (lst @ n)
;;

let makeint = function
| "" -> None
| s -> Some(Int.of_string s)
;;
```

READING CODE, REVISITED

```
f = open("data.txt")
res = []
for line in f.readlines():
    lineparts = line.split(' ')
    line_1 = []
    for part in line:
        if part == '':
            part = None
        else:
            try:
                part = int(part)
            except ValueError:
                part = None
        if part != None:
            line_1.append(int(part))
    res.append(line_1)
```

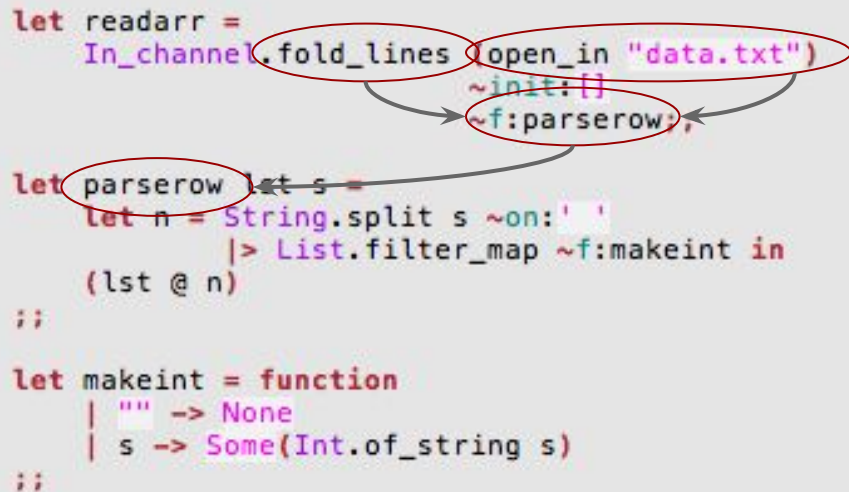
```
let readarr =
    In_channel.fold_lines (open_in "data.txt")
    ~init:[]
    ~f:parserow;

let parserow lst s =
    let n = String.split s ~on:' '
    |> List.filter_map ~f:makeint in
    (lst @ n)
;;

let makeint = function
| "" -> None
| s -> Some(Int.of_string s)
;;
```

READING CODE, REVISITED

```
f = open("data.txt")
res = []
for line in f.readlines():
    lineparts = line.split(' ')
    line_1 = []
    for part in line:
        if part == '':
            part = None
        else:
            try:
                part = int(part)
            except ValueError:
                part = None
        if part != None:
            line_1.append(int(part))
    res.append(line_1)
```



The diagram illustrates the transformation of the Python code into Erlang code. Red circles highlight specific elements in the Erlang code, and arrows show the mapping from the Python code to these elements:

- `let readarr = In_channel.fold_lines (open_in "data.txt")`: The `fold_lines` function is circled, with an arrow pointing from the `for` loop in the Python code.
- `~init:[]`: The `init` argument is circled, with an arrow pointing from the `res = []` initialization in the Python code.
- `~f:parserow;`: The `f` argument is circled, with an arrow pointing from the `for part in line:` loop in the Python code.
- `let parserow`: The `parserow` function name is circled, with an arrow pointing from the `try/except` block in the Python code.
- `let s =`: The `let s =` binding is circled, with an arrow pointing from the `lineparts = line.split(' ')` line in the Python code.
- `let n = String.split s ~on:' '`: The `let n =` binding is circled, with an arrow pointing from the `lineparts = line.split(' ')` line in the Python code.
- `|> List.filter_map ~f:makeint in`: The `filter_map` function is circled, with an arrow pointing from the `if part != None:` condition in the Python code.
- `(lst @ n)`: The `(lst @ n)` argument is circled, with an arrow pointing from the `line_1.append(int(part))` line in the Python code.

```
let readarr =
    In_channel.fold_lines (open_in "data.txt")
    ~init:[]
    ~f:parserow;

let parserow let s =
    let n = String.split s ~on:' '
    |> List.filter_map ~f:makeint in
    (lst @ n)
;;

let makeint = function
| "" -> None
| s -> Some(Int.of_string s)
;;
```

READING CODE, REVISITED

```
f = open("data.txt")
res = []
for line in f.readlines():
    lineparts = line.split(' ')
    line_1 = []
    for part in line:
        if part == '':
            part = None
        else:
            try:
                part = int(part)
            except ValueError:
                part = None
        if part != None:
            line_1.append(int(part))
    res.append(line_1)
```

```
let readarr =
    In_channel.fold_lines (open_in "data.txt")
    ~init:[]
    ~f:parserow;

let parserow let s =
    let n = String.split s ~on:' '
    > List.filter_map ~f:makeint in
    (lst @ n)
;;

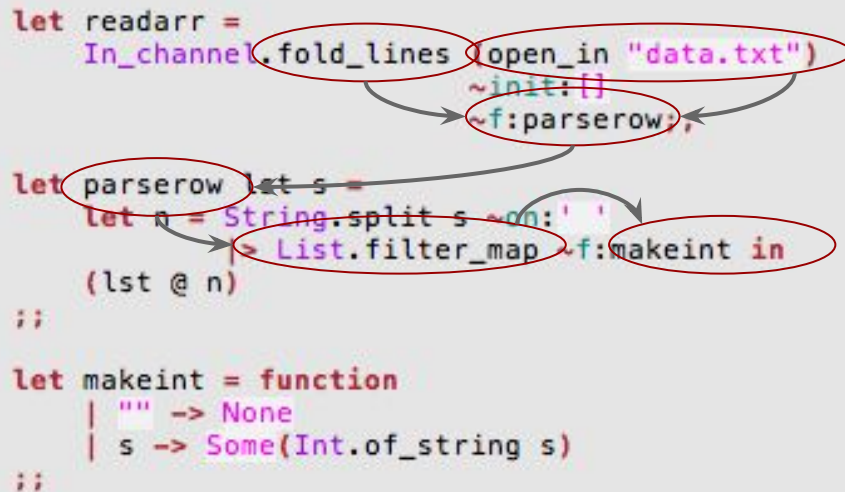
let makeint = function
| "" -> None
| s -> Some(Int.of_string s)
;;
```

The diagram illustrates the mapping between Python and Erlang code. Red ovals highlight the following elements:

- `open_in "data.txt"` in Erlang, which corresponds to `open("data.txt")` in Python.
- `~init:[]` in Erlang, which corresponds to `res = []` in Python.
- `~f:parserow` in Erlang, which corresponds to the `for part in line:` loop in Python.
- `let s =` in Erlang, which corresponds to `line` in Python.
- `String.split s ~on:' '` in Erlang, which corresponds to `line.split(' ')` in Python.
- `List.filter_map ~f:makeint` in Erlang, which corresponds to the `try/except` block in Python.
- `makeint` function in Erlang, which corresponds to the `int(part)` conversion in Python.

READING CODE, REVISITED

```
f = open("data.txt")
res = []
for line in f.readlines():
    lineparts = line.split(' ')
    line_1 = []
    for part in line:
        if part == '':
            part = None
        else:
            try:
                part = int(part)
            except ValueError:
                part = None
        if part != None:
            line_1.append(int(part))
    res.append(line_1)
```



```
let readarr =
    In_channel.fold_lines (open_in "data.txt")
    ~init:[]
    ~f:parserow;

let parserow let s =
    let n = String.split s ~on:' '
    > List.filter_map ~f:makeint in
    (lst @ n)
;;

let makeint = function
| "" -> None
| s -> Some(Int.of_string s)
;;
```

The diagram illustrates the mapping from Python to Erlang code. Red ovals highlight specific Erlang constructs: `fold_lines`, `open_in "data.txt"`, `~init:[]`, `~f:parserow;`, `let s =`, `String.split s ~on:' '`, `List.filter_map ~f:makeint in`, and `(lst @ n)`. Arrows show the flow of data and function calls between these elements.

READING CODE, REVISITED

```
f = open("data.txt")
res = []
for line in f.readlines():
    lineparts = line.split(' ')
    line_1 = []
    for part in line:
        if part == '':
            part = None
        else:
            try:
                part = int(part)
            except ValueError:
                part = None
        if part != None:
            line_1.append(int(part))
    res.append(line_1)
```

```
let readarr =
    In_channel.fold_lines (open_in "data.txt")
    ~init:[]
    ~f:parserow;

let parserow let s =
    let n = String.split s ~on:' '
    > List.filter_map ~f:makeint in
    (lst @ n)
;;

let makeint = function
| "" -> None
| s -> Some(Int.of_string s)
;;
```

The diagram illustrates the flow of data in the Erlang code. Red ovals highlight key expressions, and arrows show the flow of data between them:

- `open_in "data.txt"` is highlighted and has an arrow pointing to `~f:parserow`.
- `~init:[]` is highlighted and has an arrow pointing to `~f:parserow`.
- `~f:parserow` is highlighted and has an arrow pointing to the `let s =` binding in the `parserow` function.
- `let s =` is highlighted and has an arrow pointing to `String.split s ~on:' '`.
- `String.split s ~on:' '` is highlighted and has an arrow pointing to `List.filter_map ~f:makeint in`.
- `List.filter_map ~f:makeint in` is highlighted and has an arrow pointing to the `makeint` function definition.
- `makeint` is highlighted and has an arrow pointing to the `makeint` function definition.

READING CODE, REVISITED

- ▶ FP is explicit about concepts
 - ▷ Isolated
 - ▷ Formulated
 - ▷ Easy to associate

READING CODE, REVISITED

- ▶ FP is explicit about concepts
 - ▷ Isolated
 - ▷ Formulated
 - ▷ Easy to associate
- ▶ Understanding
 - ▷ Thinking = building a concept network

UNDERSTANDING THE CODE

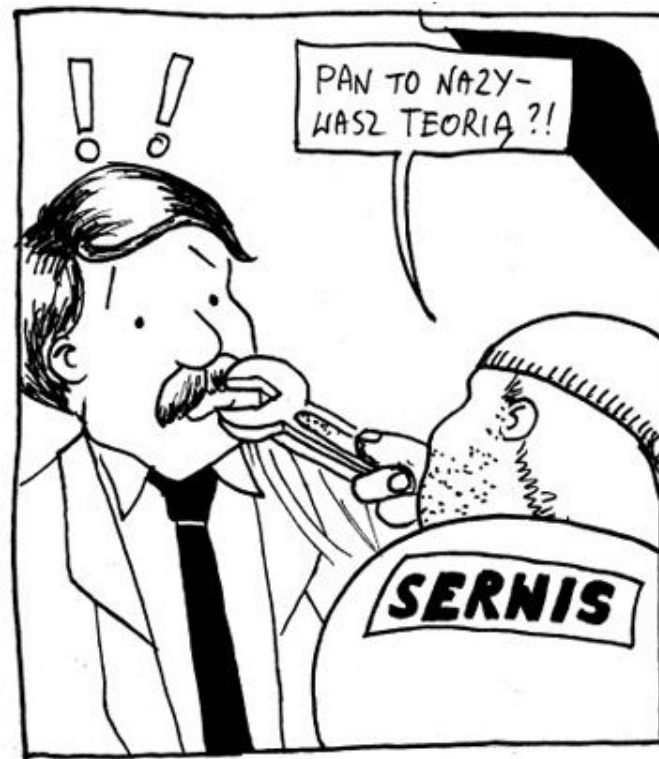


FUNCTIONAL PROGRAMMING

- ▶ Better aligned with human brain



"Coughs and stops. My theory is that a seal got stuck."



“And you call it a theory?!”



“Theory is a collection of axioms, rules of inferences and theorems derived from them. Theory is a system, not some stupid guesswork.”



LATER:

"He thinks he's so smart, while he can't tell theory from hypothesis."

IN SEARCH FOR A PROOF

- ▶ Meta-research
- ▶ Experiment

IN SEARCH FOR A PROOF

- ▶ Meta-research
- ▶ Experiment
- ▶ Which language is most readable for you?

IN SEARCH FOR A PROOF

- ▶ Meta-research
- ▶ Experiment
- ▶ Which language is most readable for you?
- ▶ Idea I: multi-language assignment
 - ▷ Reading+coding task in language to choose from
 - ▷ Measure average performance and stress level

IN SEARCH FOR A PROOF

- ▶ Meta-research
- ▶ Experiment
- ▶ Which language is most readable for you?
- ▶ Idea I: multi-language assignment
 - ▷ Reading+coding task in language to choose from
 - ▷ Measure average performance and stress level
- ▶ Idea II: natural language comparison
 - ▷ Natural language instruction
 - ▷ Written imperatively or functionally

IN SEARCH FOR A PROOF

Place the steak between two sheets of heavy plastic (resealable freezer bags work well) on a solid, level surface. Firmly pound the beef with the smooth side of a meat mallet to a thickness of 1/8 inch. Combine the olive oil, 2 tablespoons of cilantro, cumin, oregano, 1 pinch of cayenne in a large glass or ceramic bowl; season to taste with salt and pepper. Add the beef and toss to evenly coat. Cover the bowl with plastic wrap, and marinate in the refrigerator for 30 minutes.

IN SEARCH FOR A PROOF

Place the steak between two sheets of heavy plastic (resealable freezer bags work well) on a solid, level surface. Firmly pound the beef with the smooth side of a meat mallet to a thickness of 1/8 inch. Combine the olive oil, 2 tablespoons of cilantro, cumin, oregano, 1 pinch of cayenne in a large glass or ceramic bowl; season to taste with salt and pepper. Add the beef and toss to evenly coat. Cover the bowl with plastic wrap, and marinate in the refrigerator for 30 minutes.

(Prepare steak)

Prepare steak:
(wrap beef)
(pound beef)
(mix sauce)

Pound beef:
beat it with the smooth side of a meat mallet to a thickness of 1/8 inch

Mix sauce:
(get ingredients)
mix in a glass bowl
add salt and pepper

PERSONALITY

- ▶ Why do people prefer a certain programming style?
 - ▷ Accident?
 - ▷ Personality traits?
 - ▷ Attitudes?
- ▶ Requires an extensive research
 - ▷ Comparative
 - ▷ respondents?

THANK YOU FOR ATTENTION